



UNIVERSITY OF PISA

DEPARTMENT OF COMPUTER SCIENCE

Computational Mathematics

Project 17 NO-ML

Group:

Tommaso Di Vito

Francesco Romeo

ACADEMIC YEAR 2024/2025

Contents

1	Problem Analysis	3
1.1	Function definition	3
1.2	Convexity analysis	3
1.3	Gradient computation	4
1.4	Function Behavior	5
1.5	Optimal step:	5
2	Gradient Descent	7
2.1	Reformulation	7
2.2	GD algorithm	7
2.3	Complexity analysis	7
3	Quasi-Newton Method (BFGS)	8
3.1	Reformulation	8
3.2	BFGS algorithm	9
3.3	Complexity analysis	9
3.4	L-BFGS algorithm:	11
4	Convergence Analysis	12
4.1	Steps monotonicity	12
4.2	Convergence to stationary point:	13
4.3	Convergence of exact line search Gradient Descent	15
4.4	Convergence of Exact Line Search BFGS	16
4.5	Convergence rate	16
5	Experiments	18
5.1	Results Analysis	18
5.2	Plots and Observations	19

1 Problem Analysis

1.1 Function definition

Definition 1. The **2-norm** of a matrix A is defined as:

$$\|A\|_2 = \max_{v \neq 0} \frac{\|Av\|_2}{\|v\|_2} = \max_{v \neq 0} \frac{\sqrt{(Av)^T(Av)}}{\sqrt{v^T v}}$$

where $A \in \mathbb{R}^{m \times n}$, $v \in \mathbb{R}^n$.

To solve this problem, we can optimize the square of the function, since the terms under the roots are always ≥ 0 :

$$\|A\|_2^2 = \max_{v \neq 0} \frac{\|Av\|_2^2}{\|v\|_2^2} = \max_{v \neq 0} \frac{v^T A^T A v}{v^T v}$$

1.2 Convexity analysis

The function is not convex or concave.

Proof.

Definition 2. A function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is **convex** on a convex domain D if, for all $x, y \in D$ and for all $\theta \in [0, 1]$, the Jensen's inequality holds:

$$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y), \quad \forall x, y \in D, \forall \theta \in [0, 1]$$

.

Definition 3. A function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is **concave** on a convex domain D if, for all $x, y \in D$ and for all $\theta \in [0, 1]$, the following inequality holds:

$$f(\theta x + (1 - \theta)y) \geq \theta f(x) + (1 - \theta)f(y), \quad \forall x, y \in D, \forall \theta \in [0, 1]$$

.

Counterexample to Convexity:

Let $A = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$, $F_A(x) = \frac{x^T A x}{x^T x}$, $x = \begin{bmatrix} 1 \\ 0.1 \end{bmatrix}$, $y = \begin{bmatrix} 1 \\ -0.1 \end{bmatrix}$, $\theta = 0.5$. Then,

$$F_A(x) \approx \frac{2 + 0.01}{1 + 0.01} \approx 1.9901, \quad F_A(y) \approx 1.9901,$$

$$\theta F_A(x) + (1 - \theta)F_A(y) \approx 1.9901.$$

Let $z = \theta x + (1 - \theta)y = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, $F_A(z) = \frac{2}{1} = 2$. Since $F_A(z) = 2 > 1.9901$, the Jensen inequality for convexity is violated.

Counterexample to Concavity:

Let $A = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$, $x = \begin{bmatrix} 1 \\ 0.2 \end{bmatrix}$, $y = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$, $\theta = 0.5$. then

$$F_A(x) \approx \frac{2 + 0.04}{1 + 0.04} \approx \frac{2.04}{1.04} \approx 1.9615, \quad F_A(y) = \frac{2 + 4}{1 + 4} = \frac{6}{5} = 1.2,$$

$$\theta F_A(x) + (1 - \theta)F_A(y) \approx 0.5(1.9615) + 0.5(1.2) \approx 1.58075.$$

Let $z = \theta x + (1 - \theta)y = \begin{bmatrix} 1 \\ \frac{0.2+2}{2} \end{bmatrix} = \begin{bmatrix} 1 \\ 1.1 \end{bmatrix}$, then $F_A(z) \approx \frac{2+1.21}{1+1.21} \approx \frac{3.21}{2.21} \approx 1.4525$. Since $F_A(z) = 1.4525 < 1.58075$, the Jensen inequality for concavity is violated. $\square$

1.3 Gradient computation

Consider the function to be derived:

$$f(v) = \frac{v^T A^T A v}{v^T v} = \frac{v^T Q v}{v^T v}$$

where $v \in \mathbb{R}^n$, $A \in \mathbb{R}^{m \times n}$, and $Q \in \mathbb{R}^{n \times n}$ is symmetric and positive semidefinite.

$$f(v) = \frac{\sum_{i=1}^n \sum_{j=1}^n v_i Q_{ij} v_j}{\sum_{k=1}^n v_k^2}.$$

Numerator:

$$N(v) = v^T Q v = \sum_{i=1}^n \sum_{j=1}^n v_i Q_{ij} v_j.$$

Taking the derivative with respect to v_p :

$$\frac{\partial N}{\partial v_p} = \sum_{j=1}^n Q_{pj} v_j + \sum_{i=1}^n v_i Q_{ip}.$$

Since the matrix Q is symmetric, i.e. $Q_{pj} = Q_{jp}$, we can write:

$$\frac{\partial N}{\partial v_p} = 2 \sum_{j=1}^n Q_{pj} v_j.$$

Denominator:

$$D(v) = v^T v = \sum_{k=1}^n v_k^2.$$

Taking the derivative with respect to v_p :

$$\frac{\partial D}{\partial v_p} = 2v_p.$$

Using the quotient rule:

$$\frac{\partial f}{\partial v_p} = \frac{\frac{\partial N}{\partial v_p} D(v) - N(v) \frac{\partial D}{\partial v_p}}{D(v)^2}.$$

Substituting the values found:

$$\frac{\partial f}{\partial v_p} = \frac{(2 \sum_{j=1}^n Q_{pj} v_j)(v^T v) - (v^T Q v)(2v_p)}{(v^T v)^2}.$$

Writing the result in vector form, we obtain:

$$\nabla f(v) = \frac{2 (Qv(v^T v) - v(v^T Qv))}{(v^T v)^2}.$$

1.4 Function Behavior

Critical points: Setting $\nabla f(v) = 0$, we obtain:

$$Qv(v^T v) = v(v^T Qv).$$

Since $(v^T v)$ and $(v^T Qv)$ are scalars, we can rewrite the equation as:

$$Qv = \lambda v, \quad \text{where } \lambda = \frac{v^T Qv}{v^T v}.$$

Thus, the critical points of $f(v)$ are the eigenvectors of Q . Since we know that the function change convexity and the critical points correspond to the eigenvalues of Q , we can observe that:

- The function $f(v)$ is bounded between its minimum and maximum eigenvalues: $\lambda_{\min} \leq f(v) \leq \lambda_{\max}$.
- The global minimum occurs at the eigenvector corresponding to $\lambda_{\min}$.
- The global maximum occurs at the eigenvector corresponding to $\lambda_{\max}$.
- For any interior eigenvalue λ_i (i.e., $\lambda_{\min} < \lambda_i < \lambda_{\max}$), the corresponding eigenvectors are saddle points. This is because along the eigenvector direction corresponding to λ_i , small perturbations in v increase $f(v)$ if moving toward $\lambda_{\max}$ and decrease $f(v)$ if moving toward $\lambda_{\min}$.
- Observation: Along the multiples of each eigenvector we have the same value (eigenvalue).

Visualization :

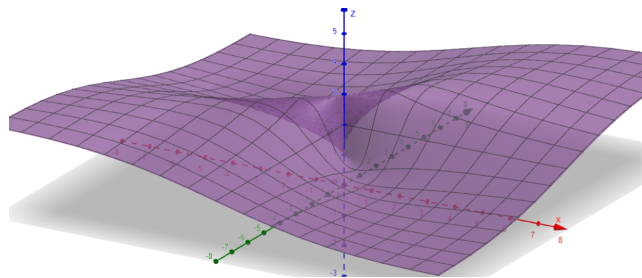


Figure 1: $A^T A$ 2x2 generic case

Optimization: We will use Gradient Descent and BFGS with exact line search for maximize our function, ensuring with a theoretical analysis that these methods will always converge to a stationary point under our assumptions.

1.5 Optimal step:

We can find the optimal step-size for gradient descent using the directional derivative. Applied to our function, we first substitute $v + \alpha d$ into the function:

$$f(\alpha) = \frac{(v + \alpha d)^T A^T A (v + \alpha d)}{(v + \alpha d)^T (v + \alpha d)}$$

Expanding this product and defining:

$$M = A^T A, \quad p = v^T v, \quad q = 2v^T d, \quad r = d^T d$$

$$a = v^T M v, \quad b = 2v^T M d, \quad c = d^T M d$$

We can rewrite the function as:

$$f(\alpha) = \frac{a + b\alpha + c\alpha^2}{p + q\alpha + r\alpha^2}$$

To find the optimal step-size, we compute the derivative of $f(\alpha)$ with respect to α :

$$\frac{d}{d\alpha} f(\alpha) = \frac{d}{d\alpha} \left(\frac{a + b\alpha + c\alpha^2}{p + q\alpha + r\alpha^2} \right)$$

Using the quotient rule for derivatives, we get:

$$\frac{d}{d\alpha} f(\alpha) = \frac{(b + 2c\alpha)(p + q\alpha + r\alpha^2) - (a + b\alpha + c\alpha^2)(q + 2r\alpha)}{(p + q\alpha + r\alpha^2)^2}$$

Setting the derivative equal to zero to find the critical points:

$$(b + 2c\alpha)(p + q\alpha + r\alpha^2) - (a + b\alpha + c\alpha^2)(q + 2r\alpha) = 0$$

Expanding and simplifying, we obtain a quadratic equation in α :

$$(bp - aq) + (2cp - 2ar)\alpha + (-br + cq)\alpha^2 = 0$$

The solutions will be given by:

$$\alpha = \frac{-B \pm \sqrt{B^2 - 4AC}}{2A} \quad \text{with} \quad A = -br + cq, B = 2cp - 2ar, C = bp - aq$$

Algorithm 1 Optimal Step Size

```

1: function COMPUTEOPTIMALSTEPSIZE( $v, d, M$ )
2:    $p \leftarrow v^T v$                                 ▷ Compute  $p = v^T v$ 
3:    $q \leftarrow 2v^T d$                                 ▷ Compute  $q = 2v^T d$ 
4:    $r \leftarrow d^T d$                                 ▷ Compute  $r = d^T d$ 
5:    $a \leftarrow v^T M v$                                 ▷ Compute  $a = v^T M v$ 
6:    $b \leftarrow 2v^T M d$                                 ▷ Compute  $b = 2v^T M d$ 
7:    $c \leftarrow d^T M d$                                 ▷ Compute  $c = d^T M d$ 
8:    $A \leftarrow -br + cq$                                 ▷ Compute coefficient  $A$ 
9:    $B \leftarrow 2cp - 2ar$                                 ▷ Compute coefficient  $B$ 
10:   $C \leftarrow bp - aq$                                 ▷ Compute coefficient  $C$ 
11:  Solve quadratic equation:  $A\alpha^2 + B\alpha + C = 0$                                 ▷ Find  $\alpha$ 
12:  Choose  $\alpha$  as the solution that minimizes  $f(v + \alpha d)$                                 ▷ Select optimal  $\alpha$ 
13:  return  $\alpha$                                 ▷ Return opposite optimal step size for minimization
14: end function

```

2 Gradient Descent

2.1 Reformulation

The formulation by max implies the search for a maximum, which requires gradient ascent for its estimation. To apply gradient descent, we must first turn the maximum problem into a minimum problem:

$$\|A\|_2^2 = \max_{v \neq 0} \frac{v^T A^T A v}{v^T v} = - \min_{v \neq 0} - \frac{v^T A^T A v}{v^T v}$$

First of all we need to swap the sign of the gradient since we reformulated our problem as a minimum problem:

$$\nabla f(v) = - \frac{2 (Qv(v^T v) - v(v^T Qv))}{(v^T v)^2}.$$

2.2 GD algorithm

Algorithm 2 Gradient Descent with Optimal Step Size

```

1: procedure GRADIENT_DESCENT( $A, v_0, \text{tol}, \text{max\_iter}$ )
2:    $M \leftarrow A^T A$  ▷ Compute  $M = A^T A$ 
3:    $v \leftarrow v_0$  ▷ Initialize the eigenvector guess
4:    $\text{iter} \leftarrow 0$  ▷ Initialize iteration counter
5:   repeat
6:      $g \leftarrow \nabla f(v)$  ▷ Compute gradient
7:     if  $\|g\| \leq \text{tol}$  then
8:       break ▷ Stop if gradient norm is below tolerance
9:     end if
10:     $d \leftarrow -g$  ▷ Compute search direction (negative gradient)
11:     $\alpha \leftarrow \text{ComputeOptimalStepSize}(v, d, M)$  ▷ Compute optimal step size
12:     $v \leftarrow v + \alpha d$  ▷ Update  $v$  using step size  $\alpha$ 
13:     $\text{iter} \leftarrow \text{iter} + 1$  ▷ Increment iteration counter
14:  until  $\text{iter} \geq \text{max\_iter}$ 
15:  return  $v$  ▷ Return approximate eigenvector
16: end procedure

```

2.3 Complexity analysis

Let m, n be the dimension of the matrix A . Let κ be the number of iterations needed by the algorithm to converge to a stationary point.

Time Complexity:

- **Precomputation:**

- Compute $M = A^T A$: $O(mn^2)$.

- **Per-Iteration Cost:**

- Gradient computation: $O(n^2)$.

- Optimal step size computation: $O(n^2)$.
- Update step: $O(n)$.
- **Total per iteration:** $O(n^2)$.
- **Overall Time Complexity:**
 - $O(mn^2 + n^2 \cdot \kappa)$. with κ as number of iterations

Space Complexity:

- **Input Storage:**
 - Matrix A : $O(mn)$.
 - Vector v : $O(n)$.
- **Intermediate Storage:**
 - Matrix M : $O(n^2)$.
 - Gradient and search direction: $O(n)$.
- **Overall Space Complexity:**
 - $O(mn + n^2)$.

3 Quasi-Newton Method (BFGS)

3.1 Reformulation

We will use the formulation of the gradient as minimum problem also in this case:

$$\nabla f(v) = -\frac{2(Qv(v^T v) - v(v^T Qv))}{(v^T v)^2}.$$

3.2 BFGS algorithm

Algorithm 3 BFGS Quasi-Newton Method with Optimal Step Size

```

1: procedure BFGS( $v, \epsilon$ )
2:    $H \leftarrow I$  ▷ Initialize Hessian approximation as the identity matrix
3:   repeat
4:      $g \leftarrow \nabla f(v)$  ▷ Compute the gradient at the current point
5:     if  $\|g\| \leq \epsilon$  then
6:       break ▷ Stop if the gradient norm is below the tolerance
7:     end if
8:      $p \leftarrow -Hg$  ▷ Compute the search direction using the Hessian approximation
9:      $\alpha \leftarrow \text{ComputeOptimalStepSize}(v, p)$  ▷ Find optimal step size formula
10:     $s \leftarrow \alpha p$  ▷ Compute the step vector
11:     $v_{\text{new}} \leftarrow v + s$  ▷ Update the position
12:     $y \leftarrow \nabla f(v_{\text{new}}) - g$  ▷ Compute the gradient difference
13:     $H \leftarrow H + \frac{yy^T}{y^T s} - \frac{Hss^T H}{s^T H s}$  ▷ Update Hessian approximation using BFGS formula
14:     $v \leftarrow v_{\text{new}}$  ▷ Update the current point
15:  until false
16:  return  $v$  ▷ Return the optimized solution
17: end procedure

```

3.3 Complexity analysis

Time Complexity:

- **Precomputation:**

- Compute $M = A^T A$: $O(mn^2)$.

- **Per-Iteration Cost:**

- Gradient computation: $O(n^2)$.
- Search direction: $O(n^2)$.
- Optimal step size: $O(n^2)$.
- BFGS update: $O(n^2)$.
- **Total per iteration:** $O(n^2)$.

- **Overall Time Complexity:**

- $O(mn^2 + n^2 \cdot \kappa)$.

Space Complexity:

- **Input Storage:**

- Matrix A : $O(mn)$.
- Vector v : $O(n)$.

- **Intermediate Storage:**

- Matrix M : $O(n^2)$.
- Hessian approximation H : $O(n^2)$.
- Gradient and search direction: $O(n)$.

- **Overall Space Complexity:**

- $O(mn + n^2)$.

For large-scale problems (e.g., $n \geq 10^6$), this algorithm becomes infeasible due to excessive memory requirements to store the Hessian matrix and the computations needed to update it. To address this, we will use also **Limited-memory BFGS (L-BFGS)** method stores only a limited history of updates, reducing memory usage to $O(mn)$, where m is the history size.

3.4 L-BFGS algorithm:

Algorithm 4 L-BFGS Quasi-Newton Method with Optimal Step Size

```

1: procedure L_BFGS( $v, \epsilon, m$ )
2:   Initialize: Empty lists  $S = []$ ,  $Y = []$  ▷ Lists to store past steps and gradient differences
3:    $g \leftarrow \nabla f(v)$  ▷ Compute initial gradient
4:   repeat
5:     if  $\|g\| \leq \epsilon$  then
6:       break ▷ Stop if the gradient norm is below the tolerance
7:     end if
8:      $q \leftarrow g$  ▷ Initialize the search direction computation
9:     Two-Loop Recursion: ▷ Compute search direction using stored history
10:    for  $i = |S| - 1$  to  $0$  do ▷ First loop: apply inverse Hessian approximation
11:       $\rho_i \leftarrow 1/(Y_i^T S_i)$  ▷ Compute scaling factor
12:       $\alpha_i \leftarrow \rho_i(S_i^T q)$  ▷ Compute intermediate coefficient
13:       $q \leftarrow q - \alpha_i Y_i$  ▷ Update intermediate vector
14:    end for
15:     $H_0 \leftarrow \frac{S_{-1}^T Y_{-1}}{Y_{-1}^T Y_{-1}} I$ 
16:     $p \leftarrow H_0 q$  ▷ Initialize search direction
17:    for  $i = 0$  to  $|S| - 1$  do ▷ Second loop: apply Hessian approximation
18:       $\beta \leftarrow \rho_i(Y_i^T p)$  ▷ Compute intermediate coefficient
19:       $p \leftarrow p + S_i(\alpha_i - \beta)$  ▷ Update search direction
20:    end for
21:     $p \leftarrow -p$ 
22:     $\alpha \leftarrow \text{ComputeOptimalStepSize}(v, p)$  ▷ Find step size using optimal step size formula
23:     $s \leftarrow \alpha p$  ▷ Compute the step vector
24:     $v_{\text{new}} \leftarrow v + s$  ▷ Update the position
25:     $y \leftarrow \nabla f(v_{\text{new}}) - g$  ▷ Compute the gradient difference
26:    Update Storage:
27:    if  $|S| \geq m$  then ▷ If history exceeds memory limit, remove oldest entries
28:      Remove oldest  $S_0, Y_0$  from lists
29:    end if
30:    Append  $s$  to  $S$ , Append  $y$  to  $Y$  ▷ Store new step and gradient difference
31:     $v \leftarrow v_{\text{new}}, g \leftarrow \nabla f(v)$  ▷ Update current point and gradient
32:  until false
33:  return  $v$  ▷ Return the optimized solution
34: end procedure

```

Time Complexity:
• Per-Iteration Cost:

- Gradient computation: $O(n^2)$.
- Search direction (two-loop recursion): $O(mn)$.

- Optimal step size: $O(n^2)$.
- **Total per iteration:** $O(n^2 + mn)$.
- **Overall Time Complexity:**
 - $O((n^2 + mn) \cdot \kappa)$.

Space Complexity:

- **Input Storage:**
 - Matrix A : $O(mn)$.
 - Vector v : $O(n)$.
- **Intermediate Storage:**
 - Lists S and Y : $O(mn)$.
 - Gradient and search direction: $O(n)$.
- **Overall Space Complexity:**
 - $O(mn)$.

4 Convergence Analysis

4.1 Steps monotonicity

The norm of the iterations for gradient descent and quasi-newton are monotonic increasing. Ensuring that these iterative methods never approach origin in the non differentiable point.

Proof. We want to prove that the sequence generated by gradient descent with the given gradient satisfies:

$$\|x_{k+1}\| \geq \|x_k\| \quad \forall k > 1.$$

To prove this we will analyse the squared norm:

$$\|x_{k+1}\|^2 \geq \|x_k\|^2.$$

For gradient descent the update in the form:

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k),$$

Expanding using the update rule:

$$(x_k + \alpha_k \nabla f(x_k))^T \cdot (x_k + \alpha_k \nabla f(x_k)) \geq x_k^T x_k.$$

Expanding the dot product:

$$\|x_k\|^2 - \alpha_k x_k^T \nabla f(x_k) - \alpha_k \nabla f(x_k)^T x_k + \alpha_k^2 \|\nabla f(x_k)\|^2 \geq \|x_k\|^2$$

Since $\|x_k\|$ vanishes and the fourth term is always greater than 0, we need to prove that holds:

$$-\alpha_k x_k^T \nabla f(x_k) - \alpha_k \nabla f(x_k)^T x_k = -2\alpha_k x_k^T \nabla f(x_k) \geq 0 \quad \forall k > 1$$

Expanding for our gradient:

$$2\alpha_k x_k^T \frac{2(Qx_k(x_k^T x_k) - x_k(x_k^T Qx_k))}{(x_k^T x_k)^2} \geq 0.$$

Expanding we get:

$$2\alpha_k(f(x_k) - f(x_k)) = 0.$$

This implies that norm of iterates never decrease approaching the origin if we use $\|x_0\| > 1$. $\square$

The same approach can be used to prove that this property holds also for quasi-newton.

Proof. We want to prove that the sequence generated by quasi-newton with the given gradient satisfies:

$$\|x_{k+1}\| \geq \|x_k\| \quad \forall k > 1.$$

To prove this we will analyse the squared norm:

$$\|x_{k+1}\|^2 \geq \|x_k\|^2.$$

For quasi-newton the update in the form:

$$x_{k+1} = x_k - \alpha_k B_k^{-1} \nabla f(x_k),$$

Expanding using the update rule:

$$(x_k + \alpha_k B_k^{-1} \nabla f(x_k))^T \cdot (x_k + \alpha_k B_k^{-1} \nabla f(x_k)) \geq x_k^T x_k.$$

Expanding as we did in the previous proof we find that this statement is true if:

$$-\alpha_k x_k B_k^{-1} \nabla f(x_k) - \alpha_k \nabla B_k^{-1} f(x_k)^T x_k = -2\alpha_k B_k^{-1} \nabla f(x_k)^T x_k \geq 0 \quad \forall k > 1$$

But we already proved that $x_k^T \nabla f(x_k) = 0$, so also $\nabla f(x_k)^T x_k = 0$. This implies that iterates never approach the origin where we have the gradient that explode to infinity. $\square$

4.2 Convergence to stationary point:

Our gradient is Lipschitz continuous and line search satisfy Wolfe conditions by definition.

Proof.

Definition 4. A continuous function $g : \mathbb{R}^d \rightarrow \mathbb{R}^m$ is said to be **Lipschitz continuous** with constant L if there exists a finite constant $L > 0$ such that:

$$\|g(x) - g(y)\| \leq L\|x - y\| \quad \forall x, y \in \mathbb{R}^d.$$

Applied to our interval:

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\| \quad \forall x, y \in \mathbb{R}^d, \|x\| > 1, \|y\| > 1.$$

The gradient of $f(x)$ is:

$$\nabla f(x) = -\frac{2(Qx(x^T x) - x(x^T Qx))}{(x^T x)^2}.$$

Now we can consider the difference between the gradients at two points x and y :

$$\|\nabla f(x) - \nabla f(y)\| = 2 \left\| \frac{(Qx(x^T x) - x(x^T Qx))}{(x^T x)^2} - \frac{(Qy(y^T y) - y(y^T Qy))}{(y^T y)^2} \right\|.$$

We can rewrite each gradient separating the terms on the numerators:

$$\|\nabla f(x) - \nabla f(y)\| = 2 \left\| \frac{(Qx)}{(x^T x)} - f(x) * \frac{x}{(x^T x)} - \frac{(Qy)}{(y^T y)} + f(y) * \frac{y}{(y^T y)} \right\|.$$

Using the triangular inequality for norm we get:

$$\|\nabla f(x) - \nabla f(y)\| \leq 2 \left\| \frac{(Qx)}{(x^T x)} - \frac{(Qy)}{(y^T y)} \right\| + 2 \left\| f(x) * \frac{x}{(x^T x)} - f(y) * \frac{y}{(y^T y)} \right\|.$$

Now we can analyse each term separately:

First term: We need to check if it holds:

$$\left\| \frac{Qx}{(x^T x)} - \frac{Qy}{(y^T y)} \right\| \leq L\|x - y\|$$

Grouping for Q , we can use induced norm inequality to get:

$$\left\| \frac{Qx}{(x^T x)} - \frac{Qy}{(y^T y)} \right\| \leq \|Q\| \left\| \frac{x}{(x^T x)} - \frac{y}{(y^T y)} \right\|$$

Since $0 \leq \|Q\| \leq \lambda_{\max}$, we can now find an L constant that depends on Q that satisfy the inequation.

Second term: We need to check if it holds:

$$\left\| f(x) * \frac{x}{(x^T x)} - f(y) * \frac{y}{(y^T y)} \right\| \leq L\|x - y\|$$

Since $0 \leq f(z) \leq \lambda_{\max}$ we can now find an L constant that depends on $f(x)$ (that depends on Q) that satisfy the inequation. $\square$

We observe that the maxima, minima, and saddle points are not isolated points; rather, they correspond to eigenvectors, meaning that every point along the direction of an eigenvector assumes the same function value. Based on this observation, we can conclude that the iterates will always increase the norm of the vector and the method will converge to a multiple of the eigenvector corresponding to the stationary point. Moreover if the iterative process starts from a point $\|x_0\| > 1$ the methods do not approach the origins where we have non differentiability and gradient explode to ∞ , the condition for Lipschitz continuity remains valid and the Wolfe conditions are satisfied by the line search.

4.3 Convergence of exact line search Gradient Descent

Assumptions:

1. The function $f(x)$ is twice continuously differentiable and bounded below.
2. The gradient $g(x) = \nabla f(x)$ is Lipschitz continuous, i.e., there exists $L > 0$ such that:

$$\|g(x) - g(y)\| \leq L\|x - y\|, \quad \forall x, y.$$

3. The update rule for gradient descent is given by:

$$x_{k+1} = x_k + \alpha_k d_k, \quad \text{where } d_k = -g_k.$$

4. The step size α_k satisfy Wolfe conditions.

To prove the global convergence of gradient descent with exact line search we can use [1] theorem 3.2 that states:

Theorem 1. *Consider any iteration of the form $x_{k+1} = x_k + \alpha_k p_k$, where p_k is a descent direction and α_k satisfies the Wolfe conditions. Suppose that f is bounded below in $\mathbb{R}^n$ and that f is continuously differentiable in an open set $\mathcal{N}$ containing the level set*

$$\mathcal{L} \stackrel{\text{def}}{=} \{x : f(x) \leq f(x_0)\},$$

where x_0 is the starting point of the iteration. Assume also that the gradient ∇f is Lipschitz continuous on $\mathcal{N}$, that is, there exists a constant $L > 0$ such that

$$\|\nabla f(x) - \nabla f(\tilde{x})\| \leq L\|x - \tilde{x}\|, \quad \text{for all } x, \tilde{x} \in \mathcal{N}.$$

Then,

$$\sum_{k \geq 0} \cos^2 \theta_k \|\nabla f_k\|^2 < \infty.$$

This is called The Zoutendijk condition and it implies that:

$$\cos^2 \theta_k \|\nabla f_k\|^2 \rightarrow 0.$$

This limit can be used in turn to derive global convergence results for line search algorithms. If our method for choosing the search direction p_k in the iteration $x_{k+1} = x_k + \alpha_k p_k$ ensures that the angle θ_k defined by $\cos \theta_k = \frac{-\nabla f_k^T p_k}{\|\nabla f_k\| \|p_k\|}$ is bounded away from 90° , there is a positive constant δ such that

$$\cos \theta_k \geq \delta > 0, \quad \text{for all } k.$$

It follows immediately from that

$$\lim_{k \rightarrow \infty} \|\nabla f_k\| = 0.$$

In other words, we can be sure that the gradient norms $\|\nabla f_k\|$ converge to zero, provided that the search directions are never too close to orthogonality with the gradient. In particular, the method of steepest descent (for which the search direction p_k is parallel to the negative gradient) produces a gradient sequence that converges to zero, provided that it uses a line search satisfying the Wolfe condition.

This theorem holds for our case since we have gradient descent with exact line search that satisfy Wolfe conditions and so it produces a gradient sequence that converge to 0 indicating that the iterates converge to a point x^* such that the gradient is 0 (stationary point).

4.4 Convergence of Exact Line Search BFGS

Assumptions

1. The objective function f is twice continuously differentiable.
2. The level set $L \subseteq \{x \in \mathbb{R}^n \mid f(x) \leq f(x_0)\}$ is convex, and there exist positive constants m and M such that

$$m \leq z^T G(x) z \leq M \quad \text{for all } z \in \mathbb{R}^n \text{ and } x \in L.$$

Part (ii) of this assumption implies that $G(x)$ is positive definite on L and that f has a unique minimizer x^* in L .

Now to prove the convergence of BFGS we can use theorem 6.5 [1] which states:

Theorem 2. *Let B_0 be any symmetric positive definite initial matrix, and let x_0 be a starting point for which Assumption 6.1 is satisfied. Then the sequence $\{x_k\}$ generated by Algorithm 6.1 (with $\phi = 0$) converges to the minimizer x^* of f .*

This theorem holds always for convex function, but in our case that we have a non-convex function, this theorem does not apply globally. In particular, the theorem holds if and only if the Hessian matrix is positive definite on the level set L . This condition is met when the function behaves as a convex function and iterates generate positive definite Hessian matrix.

4.5 Convergence rate

Gradient descent: The rate of convergence of gradient descent will be given by the theorem [1] 3.4 that states:

Theorem 3. *Suppose that $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is twice continuously differentiable, and that the iterates generated by the steepest-descent method with exact line searches converge to a point x^* at which the Hessian matrix $\nabla^2 f(x^*)$ is positive definite. Let r be any scalar satisfying*

$$r \in \left(\frac{\lambda_n - \lambda_1}{\lambda_n + \lambda_1}, 1 \right),$$

where $\lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$ are the eigenvalues of $\nabla^2 f(x^*)$. Then for all k sufficiently large, we have

$$f(x_{k+1}) - f(x^*) \leq r^2 [f(x_k) - f(x^*)].$$

When iterates approach the point x^* , the rate of convergence is influenced by the gap between the eigenvalues of the Hessian matrix at x^* . A larger gap in the eigenvalues typically results in slower convergence. This result is valid for minimum points, where the Hessian is positive definite. However, we cannot make similar conclusions for saddle points and for cases where the Hessian is not positive definite. Sometimes the Hessian may have zero as an eigenvalue. In such cases, our theorem does not hold. We will use the formulation as maximum problem to show that in some cases in point of maximum eigenvalue the Hessian is semidefinite negative, but this is true also for minimum formulation swapping the sign (semidefinite positive):

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} x = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$f(x) = \frac{x^T A x}{x^T x}$$

$$H(f(x)) = \frac{2A}{x^T x} - \frac{4Axx^T A}{(x^T x)^2} - 2f(x) \left(\frac{I}{x^T x} - \frac{2xx^T}{(x^T x)^2} \right)$$

$$H(x) = \begin{bmatrix} 0 & 0 \\ 0 & -2 \end{bmatrix} \Rightarrow \lambda_{\min} = -2, \lambda_{\max} = 0$$

This happens when in the point of maximum/minimum small changes to the vector in some directions don't produce significant changes to the value of the function. In particular this happens when the eigenvector has all zeros as components and a value different from 0. In this case moving along the direction of the value different from 0 we get a multiple of the eigenvector and so the function value remains the same (eigenvalue). We can say that it happens when the eigenvector is parallel to an axis and perpendicular to all the others.

BFGS: The rate of convergence of quasi-newton methods instead will be given by theorem [1] 6.6 that states:

Theorem 4. *Suppose that f is twice continuously differentiable and that the iterates generated by the BFGS algorithm converge to a minimizer x^* at which the Hessian matrix G is Lipschitz continuous at x^* . Suppose also that $\sum_{k=1}^{\infty} \|x_k - x^*\| < \infty$ holds. Then x_k converges to x^* at a super-linear rate.*

When the iterates approach the point x^* , and assuming that the Hessian matrix is Lipschitz continuous in a neighbourhood of this point, the algorithm converges with a super-linear rate to the solution. To verify this, we need to check whether the Lipschitz continuity condition holds for the Hessian:

$$\|\nabla^2 f(x) - \nabla^2 f(x^*)\| \leq L\|x - x^*\|, \quad \forall x \text{ in a neighbourhood of } x^*. \quad (1)$$

To analyse this, we use the first-order Taylor expansion:

$$\|\nabla^2 f(x) - \nabla^2 f(x^*)\| \approx \|\nabla^3 f(\xi)\| \cdot \|x - x^*\|, \quad (2)$$

where ξ is some point in the neighbourhood of x^* . Therefore, we need to verify the boundedness condition:

$$\|\nabla^3 f(\xi)\| \leq L. \quad (3)$$

Our goal is to check whether the third-order derivative tensor is bounded in the neighbourhood of x^* . If this condition holds, the Hessian will be Lipschitz continuous, and consequently, the algorithm will converge super-linearly in the neighbourhood of x^* . We consider the Hessian of the function, which (omitting derivation details) can be written as

$$H(f(x)) = \frac{2A}{x^T x} - \frac{4Axx^T A}{(x^T x)^2} - 2f(x) \left(\frac{I}{x^T x} - \frac{2xx^T}{(x^T x)^2} \right)$$

We observe that the numerator has a maximum polynomial degree of 2 (e.g., in terms like $Axx^T A$), while the denominator reaches degree 4 (e.g., $(x^T x)^2$). To analyse the third-order derivative, we apply the quotient rule:

$$\frac{\partial f}{\partial v_p} = \frac{\frac{\partial N}{\partial v_p} D(v) - N(v) \frac{\partial D}{\partial v_p}}{D(v)^2}.$$

- For the first term: $\partial N / \partial v_p \cdot D(v)$, the derivative reduces the degree of the numerator to 1 and multiplies it by a denominator of degree 4, resulting in an overall numerator of degree 5.
- For the second term: $N(v) \cdot \partial D / \partial v_p$, the numerator (degree 2) is multiplied by a derivative of the denominator (degree 3), again resulting in a total degree of 5.

Thus, the third derivative has a numerator of maximum degree 5 and a denominator of degree 6. Now, since we are working under the condition $\|x_0\| > 1$, the function is bounded at least for values less than ∞ . Moreover, as $\|x\| \rightarrow \infty$, the denominator dominates the numerator (degree 6 vs. 5), so the entire expression remains bounded.

5 Experiments

Goal: The objectives of our experiments are:

- Compare the performance of the algorithms in terms of execution time to determine which performs better and under which conditions.
- Compare the results found with the real expected value.
- Examine how computational time and number of iterations to converge scales with increasing matrix size.
- Analyse the impact of different starting vectors.

Algorithms: For our experiments, we will use Gradient Descent and BFGS, as required by the problem. Additionally, we will use L-BFGS to compare its performance with standard BFGS.

Setup: We will run our experiments on a machine with the following specifications:

- Intel Core i7 (8th Gen) CPU.
- 8GB RAM.

5.1 Results Analysis

We evaluated the performance of three optimization algorithms Gradient Descent, BFGS, and L-BFGS on matrices of varying sizes. All algorithms were run under controlled conditions, with a fixed maximum number of iterations and a fixed gradient tolerance. For each matrix size, we used the same randomly generated matrix across all algorithms, ensuring that the matrix had the same maximum eigenvalue regardless of its size. We also did multiple trials for all the sizes of the matrices, fixing the input matrix for all the trials (same matrix for all trials) and using different randomly generated starting vectors (different starting positions).

- Random initial vector v_0 with $\|v_0\| > 1$ generated from random Gaussian distribution.
- Random matrix A such that $A^T A$ has random distribution over the eigenvalues and $\lambda_{\max} = 100$.

- Matrix sizes: All matrices of size $N \times M$ with $N, M \in [250, 1000, 2500]$.
- 5 independent trials per matrix size (each with a different random matrix and vector).
- 300 maximum number of iterations for all algorithms.
- 1×10^{-6} gradient norm tolerance for early stopping.
- 10 history size for L-BFGS.

To create our random vector with $\|x_0\| > 1$ we used the following function:

```

1: function RANDOMSTARTVECTOR( $n, r_{\min}$ )
2:    $v \leftarrow$  Random vector from  $\mathbb{R}^n$  with standard Gaussian distribution.
3:    $v \leftarrow v/\|v\|$  ▷ Normalize to unit norm
4:    $r \leftarrow$  Uniform sample from  $[r_{\min}, 2r_{\min}]$ 
5:    $v \leftarrow r \cdot v$  ▷ Scale to desired norm range
6:   return  $v$ 
7: end function

```

To create our input matrix $A^T A$ for our function with $\lambda_{\max} = 100$ we used the following function:

```

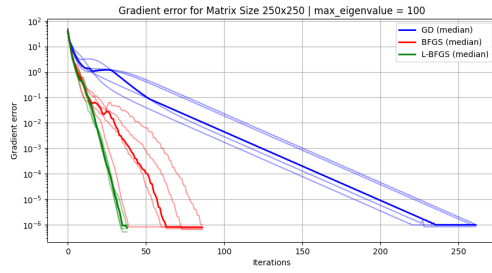
1: function GENERATEMATRIX( $n, m, \lambda_{\max}$ )
2:    $A \leftarrow$  Random matrix of size  $n \times m$ 
3:    $M \leftarrow A^T A$  ▷ Symmetric positive semi-definite matrix
4:    $\lambda_{\max}^{\text{orig}} \leftarrow$  Largest eigenvalue of  $M$ 
5:    $s \leftarrow \sqrt{\lambda_{\max}/\lambda_{\max}^{\text{orig}}}$ 
6:    $A_{\text{scaled}} \leftarrow s \cdot A$ 
7:   return  $A_{\text{scaled}}^T A_{\text{scaled}}$  ▷ Return  $A^T A$ 
8: end function

```

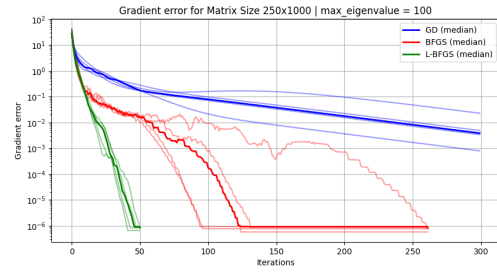
Note: The algorithms implementations in our python code assume that the input matrix is already symmetric positive definite $A^T A$. We precomputed this product before starting all the algorithms directly after generating the random matrix of size $N \times M$ to avoid to recompute the product every time making the algorithms much more efficient.

5.2 Plots and Observations

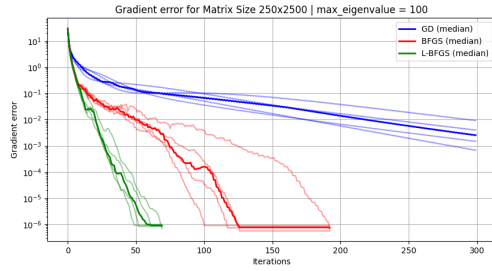
Gradient Error Over Iterations: The following plots illustrate the gradient error across iterations for each matrix size. In each plot, the **bold line** represents the **median** behaviour across the three trials, while the fainter lines (lower **alpha** values) represent individual trial runs:



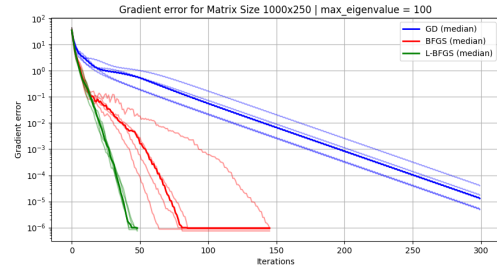
Gradient error 250x250



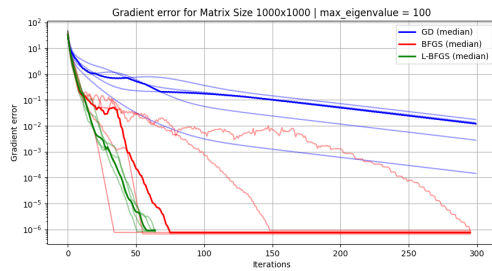
Gradient error 250x1000



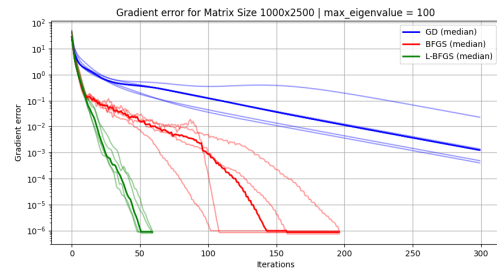
Gradient error 250x2500



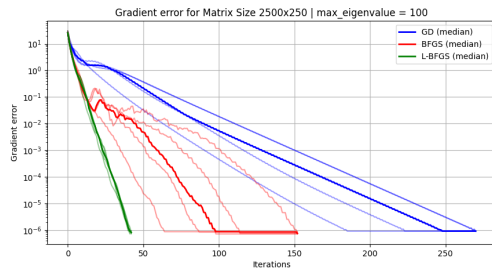
Gradient error 1000x250



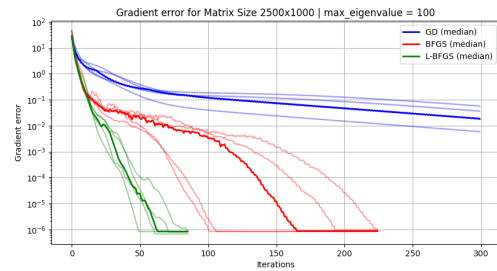
Gradient error 1000x1000



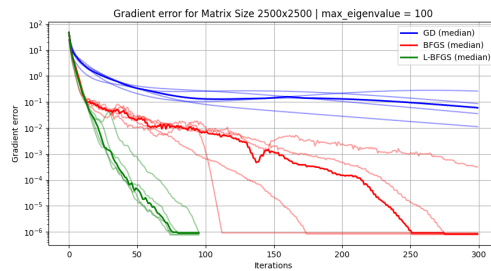
Gradient error 1000x2500



Gradient error 2500x250



Gradient error 2500x1000



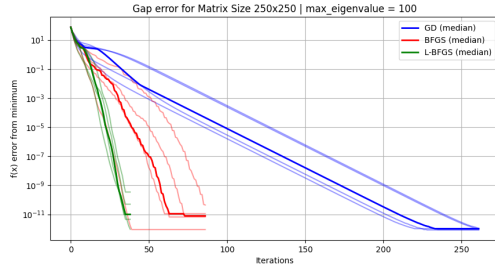
Gradient error 2500x2500

From the plots, we observe:

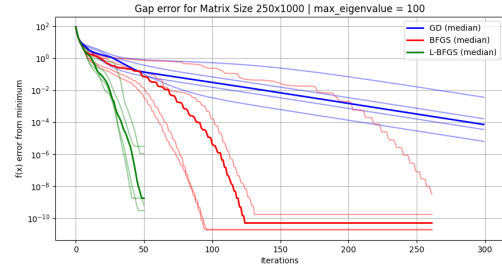
- High consistency across trials, with very similar trends.

- L-BFGS and BFGS significantly reduce the gradient error faster than Gradient Descent.
- Although BFGS and L-BFGS follow similar patterns, L-BFGS consistently converges more quickly.
- L-BFGS Converges faster than BFGS almost every time it converges to a saddle point, while BFGS converges almost always to the real minimum (-100) for our experiments.

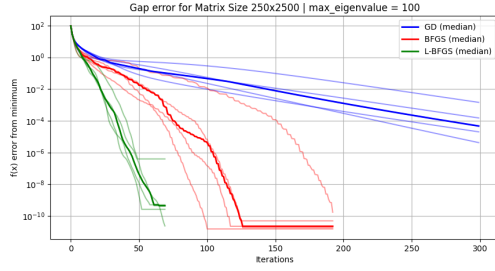
Distance error from minimum over Iterations: The following plots illustrate the distance from the real minimum over the iteration. In **bold** we have the **median** behaviour and faint lines represent individual runs:



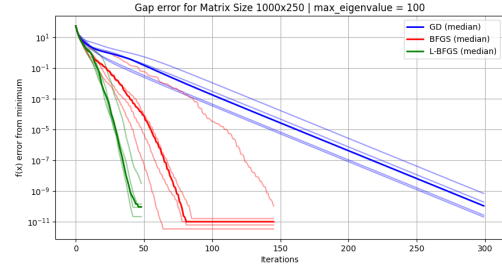
Gap from minimum 250x250



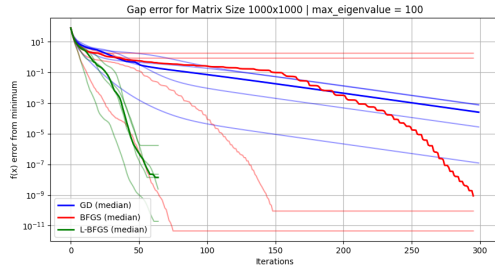
Gap from minimum 250x1000



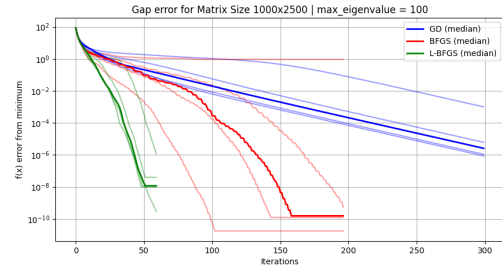
Gap from minimum 250x2500



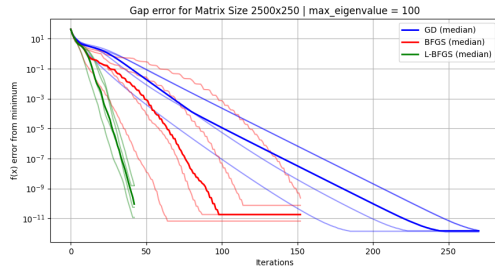
Gap from minimum 1000x250



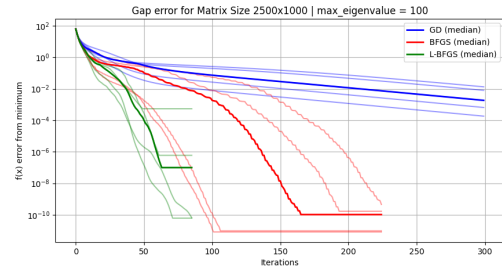
Gap from minimum 1000x1000



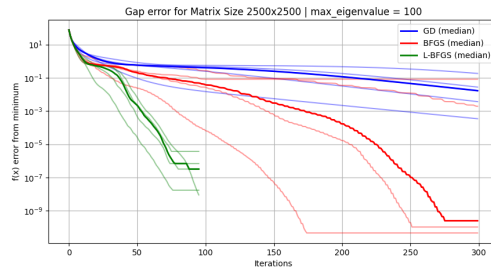
Gap from minimum 1000x2500



Gap from minimum 2500x250



Gap from minimum 2500x1000



Gap from minimum 2500x2500

From the plots, we observe:

- The gap error decrease monotonically suggesting the algorithm is well implemented. From

plots sometimes can be hard to see for BFGS that makes jumps, but checking programmatically we established that the value of the function between consecutive steps of individual trials always decrease.

- BFGS and L-BFGS gets always lower error in very few iteration than gradient descent that instead requires many iterations to achieves a low error.
- L-BFGS achieves very low error in very few iteration.
- Most of the times all the algorithms converge to the minimum, but sometimes we can see for example from the second plot (250x1000) that BFGS stops to a saddle point after ≈ 20 iterations since the gap error stops decreasing.

Time Results: The following table summarizes the average time (in seconds) required by each algorithm to converge varying the input matrix size (i.e., when the gradient norm fell below the set tolerance or when the maximum number of iterations was reached). Standard deviations across the 5 trials are also reported:

Input size	$A^T A$ size	GD Time	GD Iter	BFGS Time	BFGS Iter	L-BFGS Time	L-BFGS Iter
250×250	250×250	0.0496 ± 0.0030	241.6 ± 15.6	0.1166 ± 0.0245	65.2 ± 17.6	0.0150 ± 0.0016	36.4 ± 1.1
250×1000	1000×1000	0.9840 ± 0.0163	300.0 ± 0.0	9.9596 ± 4.1290	142.2 ± 66.7	0.5571 ± 0.0363	46.4 ± 3.3
250×2500	2500×2500	6.7158 ± 0.0371	300.0 ± 0.0	76.6234 ± 18.2702	133.0 ± 34.5	3.8496 ± 0.4817	58.8 ± 6.9
1000×250	250×250	0.0595 ± 0.0044	300.0 ± 0.0	0.1482 ± 0.0453	91.6 ± 29.3	0.0169 ± 0.0009	45.4 ± 2.3
1000×1000	1000×1000	0.8796 ± 0.0243	300.0 ± 0.0	7.6799 ± 6.3323	122.4 ± 89.7	0.6833 ± 0.0458	59.0 ± 4.5
1000×2500	2500×2500	6.5130 ± 0.0401	300.0 ± 0.0	81.5005 ± 19.2765	142.4 ± 33.3	3.5381 ± 0.2909	54.2 ± 4.9
2500×250	250×250	0.0469 ± 0.0070	240.2 ± 33.3	0.1679 ± 0.0511	104.0 ± 31.1	0.0159 ± 0.0012	42.2 ± 0.8
2500×1000	1000×1000	0.8614 ± 0.0261	300.0 ± 0.0	10.2858 ± 3.4985	158.8 ± 48.6	0.7735 ± 0.1596	66.8 ± 13.8
2500×2500	2500×2500	6.6229 ± 0.0456	300.0 ± 0.0	129.2252 ± 42.5211	223.2 ± 74.2	5.2731 ± 0.4987	81.0 ± 7.4

Table 1: Performance comparison of optimization methods on various matrix sizes.

From the timing results, we observe:

- BFGS is significantly slower than Gradient Descent due to the cost of updating and storing the full Hessian matrix.
- L-BFGS is the fastest overall in both iteration count and execution time, as it leverages a limited memory approximation requiring less updates, but still getting more informations on the shape of the function than gradient descent.
- We underline the fact that the standard deviation of BFGS is high since it is very depend on the starting point, if the starting point is good it requires very few iterations to converge to the desiderated error and so the time required is much smaller otherwise it requires many more iterations increasing a lot the computational time.

Summarizing the results of our experiments, we observe that, as expected, Gradient Descent (GD) converges significantly more slowly in terms of iterations compared to BFGS and L-BFGS. This is because BFGS-based methods leverage Hessian approximations to better capture the curvature of the objective function. Among them, L-BFGS generally outperforms BFGS in terms of iteration count, due to its use of a limited memory approach since it only considers a small window of recent updates, which allows it to focus more effectively on the local curvature around the current point. In contrast, BFGS uses the full history of updates, which can dilute the influence of recent curvature information. However, this advantage in iteration count comes with a trade-off. L-BFGS often converges to saddle points, resulting in a higher error gap compared to BFGS, which converges to the true minimum more often. In terms of

computational time per iteration, GD is the fastest due to its simplicity, while BFGS is the slowest due to the overhead of maintaining and updating a full Hessian approximation. L-BFGS strikes the best balance overall achieving relatively fast convergence with low computational overhead and a reasonably low error gap in most cases making it the most efficient method in our experiments.

References

- [1] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, New York, NY, 2 edition, 2006.